



Transformer



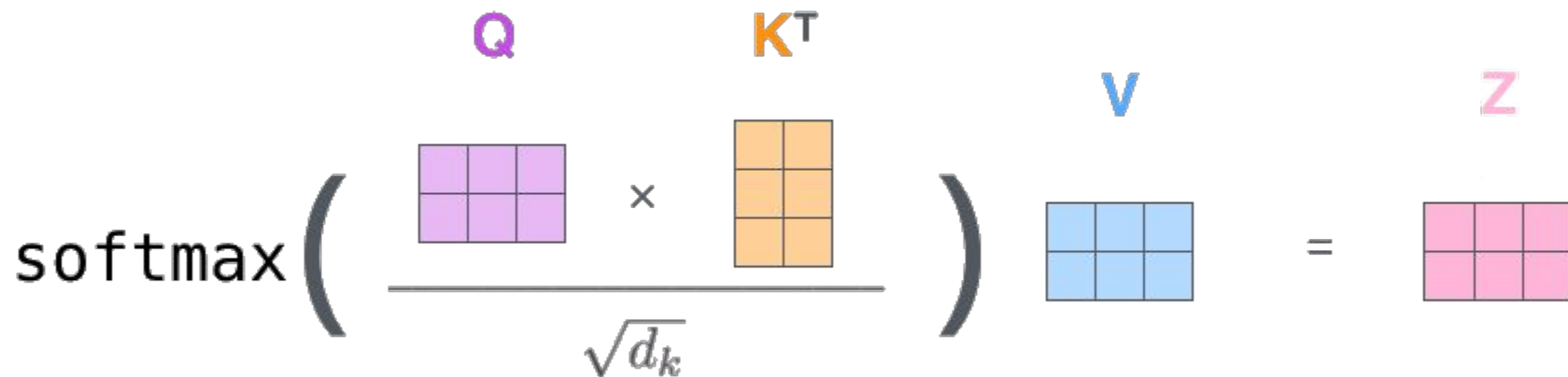
Attention

Q(Query) : (디코더에서) 현재 단어가 무엇을 찾고 있는가?

K(Key) : (인코더의) 단어가 어떤 정보를 갖고 있는가?

V(Value) : (인코더의) 각 단어가 갖고 있는 정보

Dot-Product Attention



The diagram illustrates the Dot-Product Attention mechanism. It shows the calculation of a weighted sum of values V based on the similarity between a query Q and a key K^T . The query Q is a 2x3 purple grid, and the key K^T is a 3x2 orange grid. These are multiplied and the result is divided by $\sqrt{d_k}$ (the dimension of the keys). The result is then passed through a softmax function. The value V is a 2x3 blue grid. The final output Z is a 2x3 pink grid.

$$\text{softmax} \left(\frac{Q \times K^T}{\sqrt{d_k}} \right) V = Z$$



Dot-Product Attention

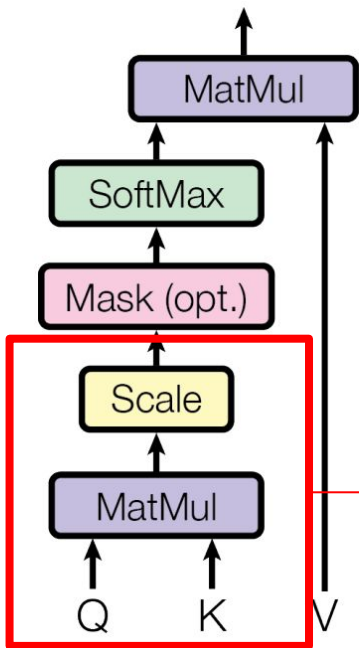
Bahdanau Attention

$$e = W_s \cdot \tanh(W_{En}H + W_{De}s_{t-1})$$

Scaled Dot-Product Attention

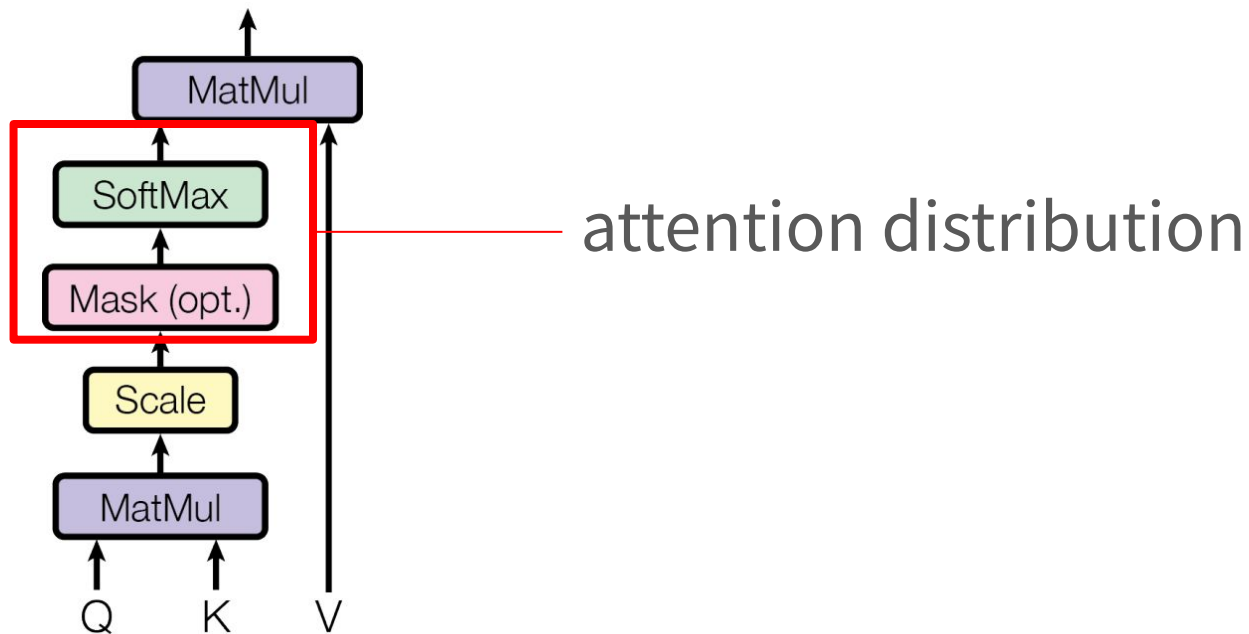
$$e = \frac{Q \cdot K^{\top}}{\sqrt{d_k}}$$

Dot-Product Attention

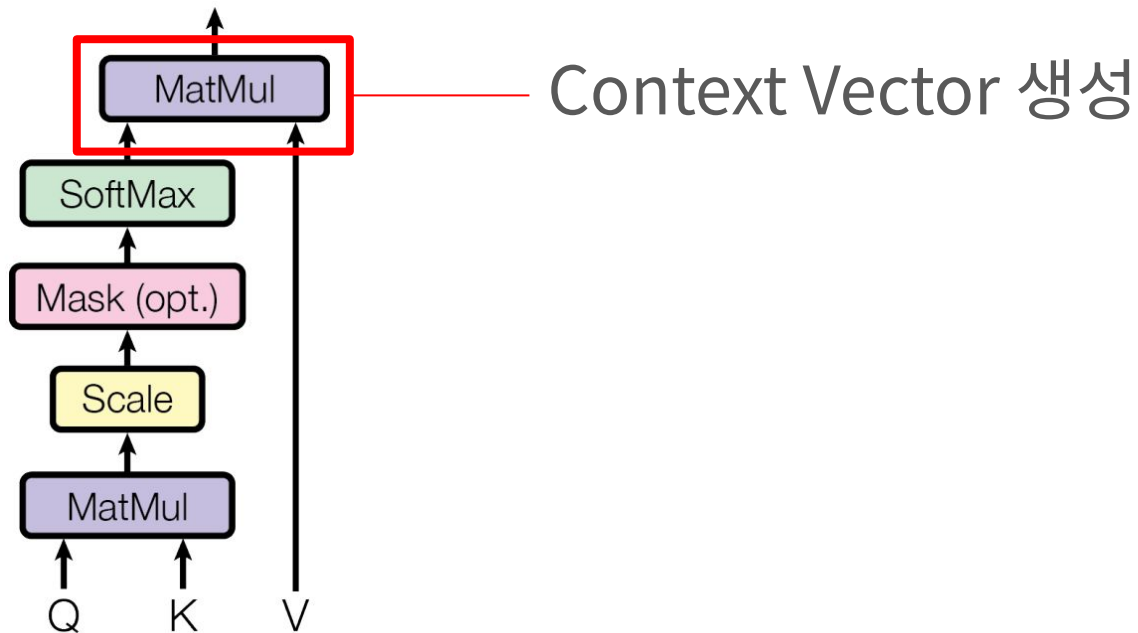


유사도 점수 계산

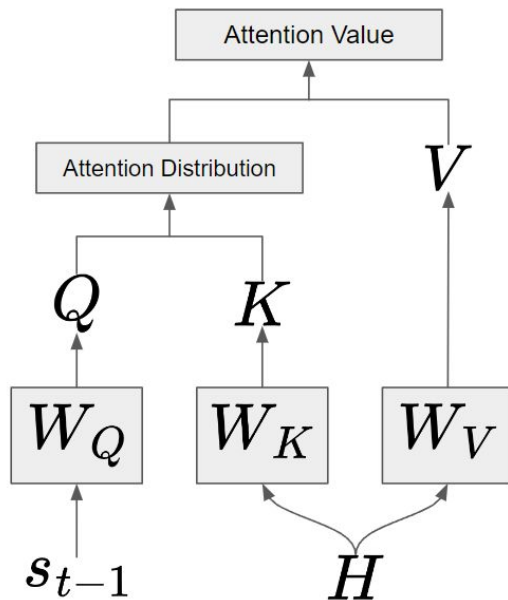
Dot-Product Attention



Dot-Product Attention



Dot-Product Attention

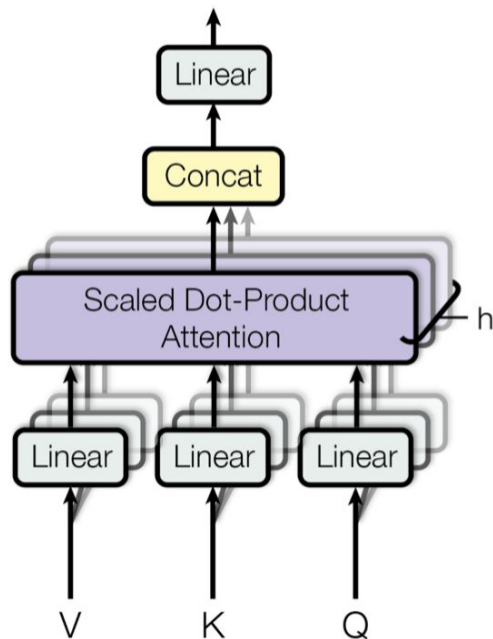


Multi-head Attention

단일 Attention으로는 다양한
문맥정보를 탐색할 수 없음

각 벡터를 N개로 분할해 각각
Attention 적용

이후 concat 후 linear를 통해 의미
통합





Multi-head Attention

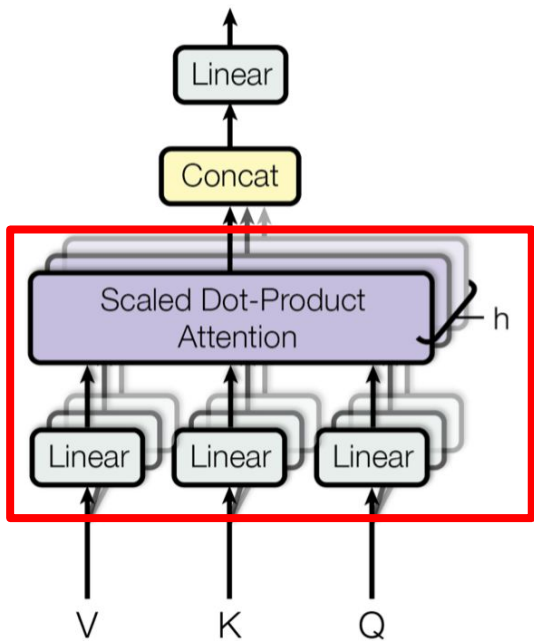
Attention 연산은 한 번에 한 가지 관계만 학습

-> 서로 다른 문맥 관계를 학습하여야함

각 head별 독립된 가중치를 가짐

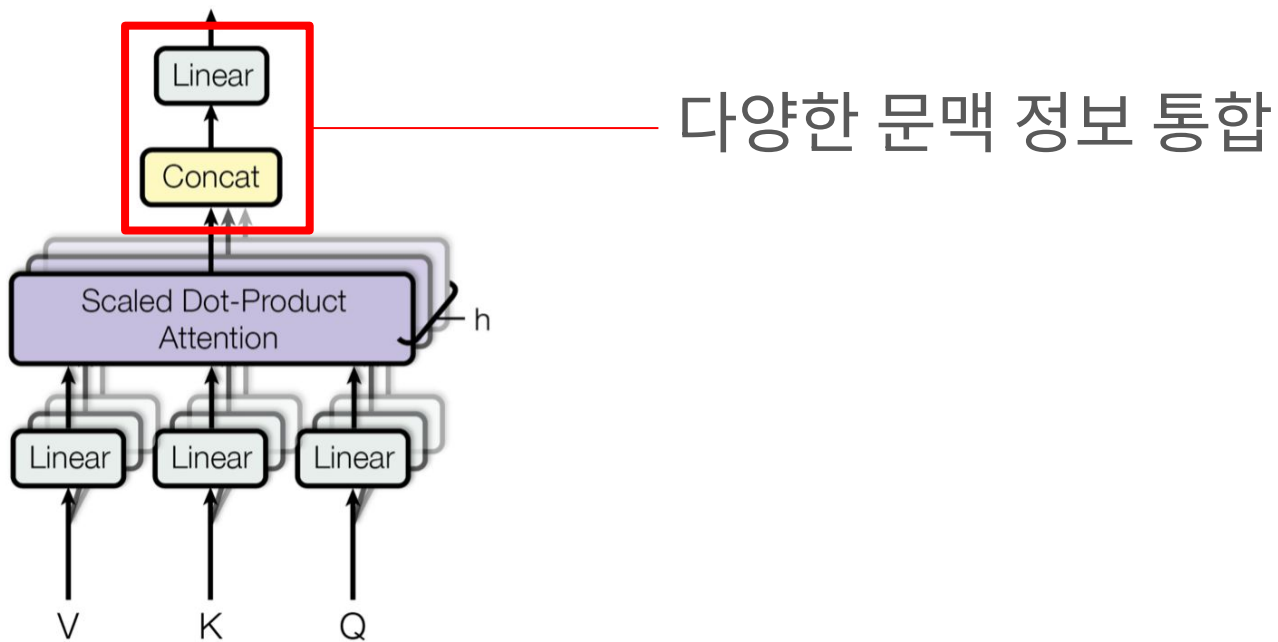
-> 서로 다른 subspace를 학습

Multi-head Attention



다양한 문맥 정보 학습

Multi-head Attention



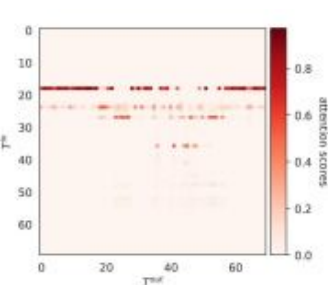


Multi-head Attention

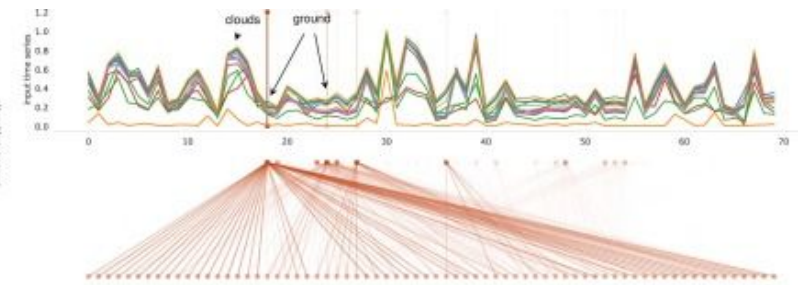
단일 Attention으로는 다양한 문맥정보를 탐색할 수
없음

각 벡터를 N개로 분할해 각각 Attention 적용

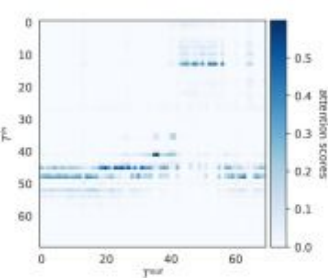
Multi-head Attention



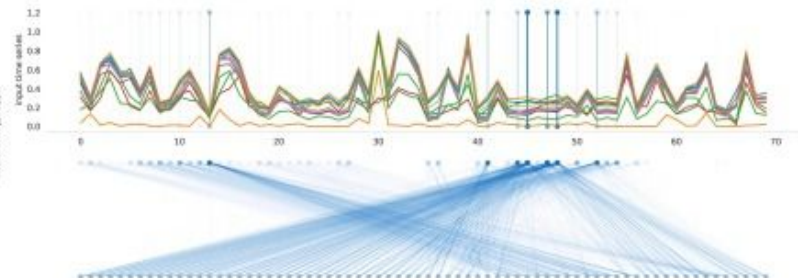
(a) Self-Attention Matrix of Head 1



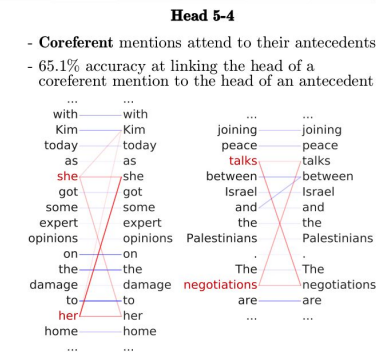
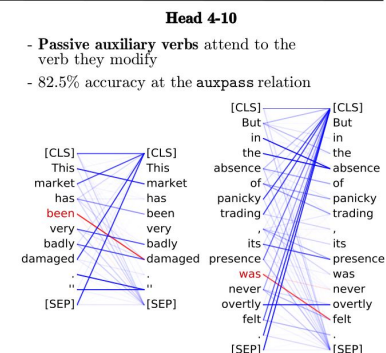
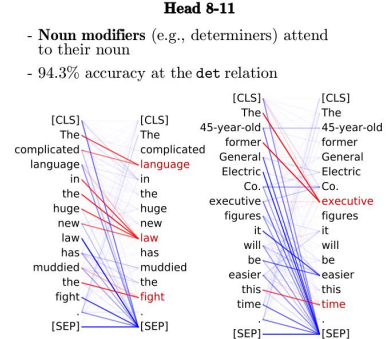
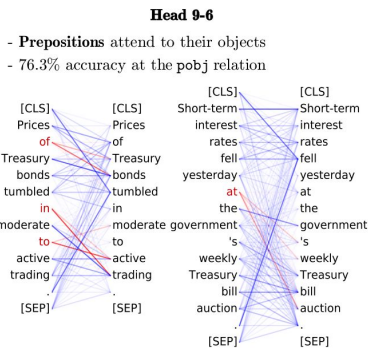
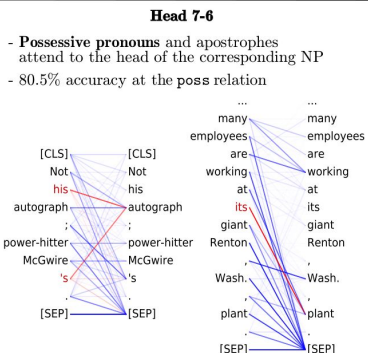
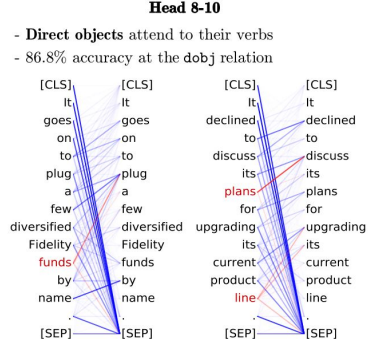
(b) Head 1 as bipartite graph visualization in context of the input time series



(c) Self-Attention Matrix of Head 2



(d) Head 2 as bipartite graph visualization in context of the input time series





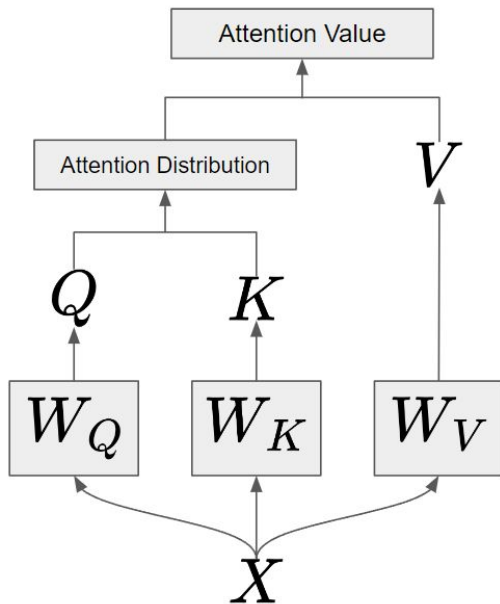
Self Attention

단어는 앞 뒤 문맥에 따라 내포하는 뉘앙스가 달라짐.

- 현재도 수많은 고래들이 남획되고 있습니다.
- 배경은 고래와 늙은 어부의 사투를 그린 작품입니다.

따라서 Word2Vec처럼 고정된 벡터 값이 아닌 각 문맥을 반영해 유동적으로 의미를 바꿀 필요가 있음

Self Attention



각 토큰 벡터가 문장 안의 다른 모든 단어를 참조하여 자기 자신의 표현을 문맥에 맞게 조정하는 과정



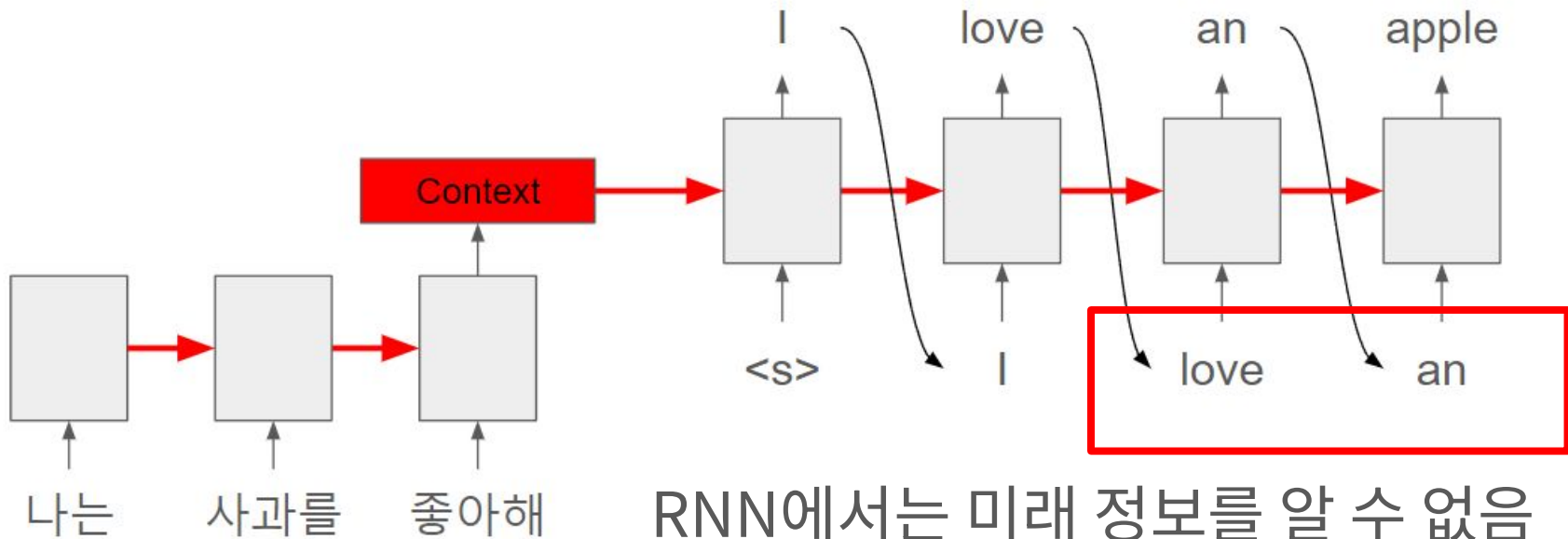
Cross Attention

Decoder에서 다음 단어를 추론할 때 Encoder의 입력 시퀀스를 참고하여 추론하여야함.

Q : Decoder의 hidden state

K, V : Encoder의 hidden state

Masked Self-Attention





Masked Self-Attention

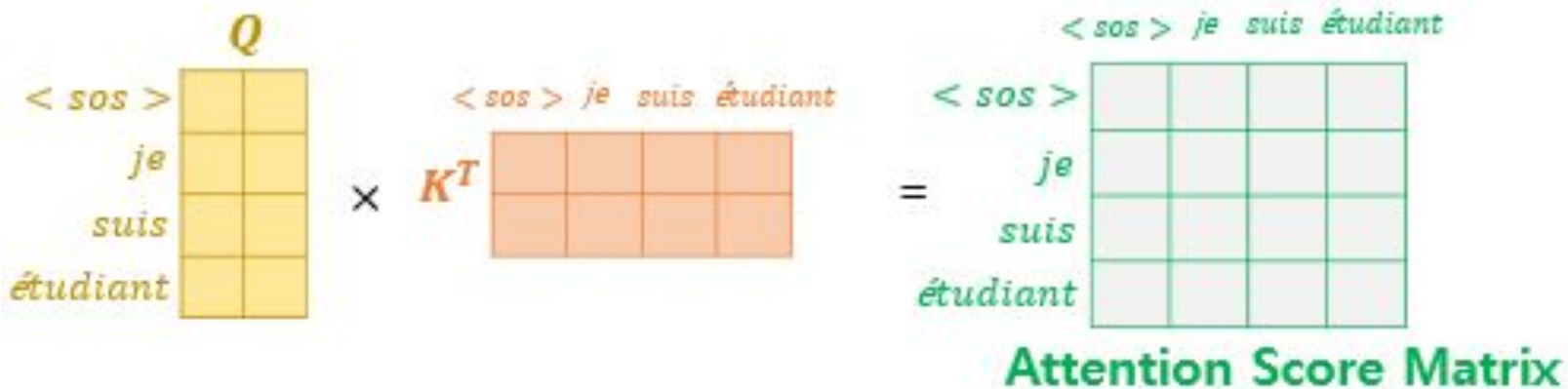
RNN 처럼 디코더의 입력을 한 토큰씩 늘려가는 것

-> 순차적 학습이 가능하지만 Attention 병렬 연산의
장점이 사라짐

모든 토큰을 입력할 경우

-> 실제 추론에서는 다음에 올 토큰을 알 수 없음

Masked Self-Attention

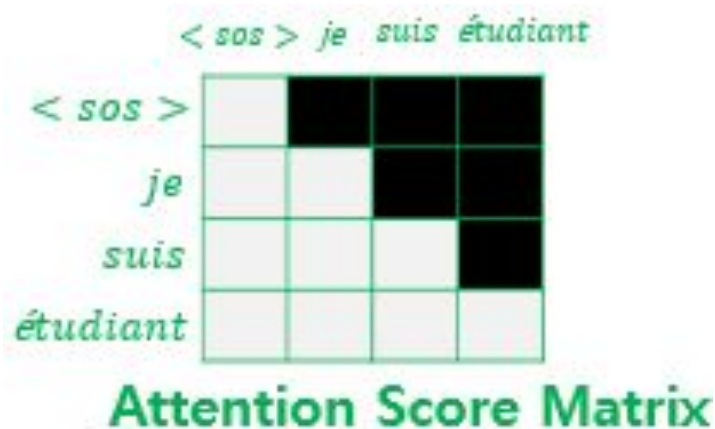


학습 단계에서 다음 토큰의 정보를 보고 연산(추론과 다름)

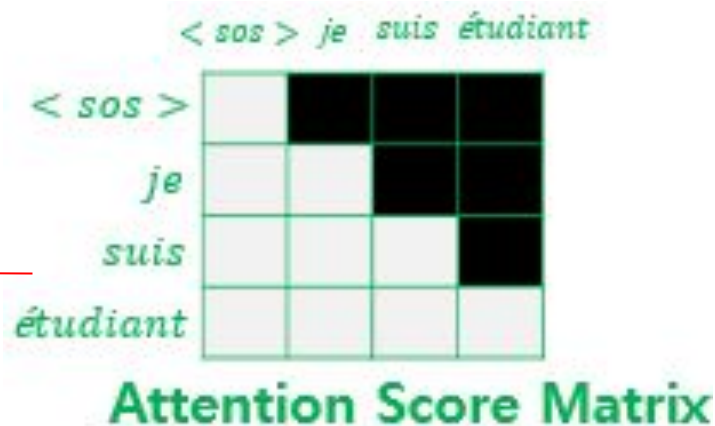
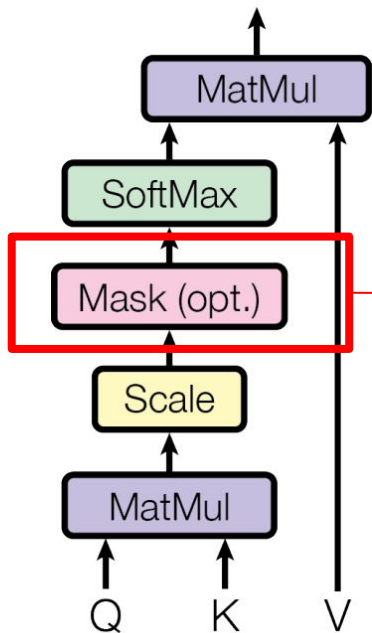
Masked Self-Attention

미래 정보를 Masking
처리해서 정보가 들어가지
않도록 함

Look-ahead Mask
(Causal Mask)



Masked Self-Attention



마이너스 무한대 값 입력



Feed-Forward Neural Network

Attention에도 linear가 있지만 이 역할은 다양한 문맥 정보를 하나의 통합된 표현으로 정리하는 역할

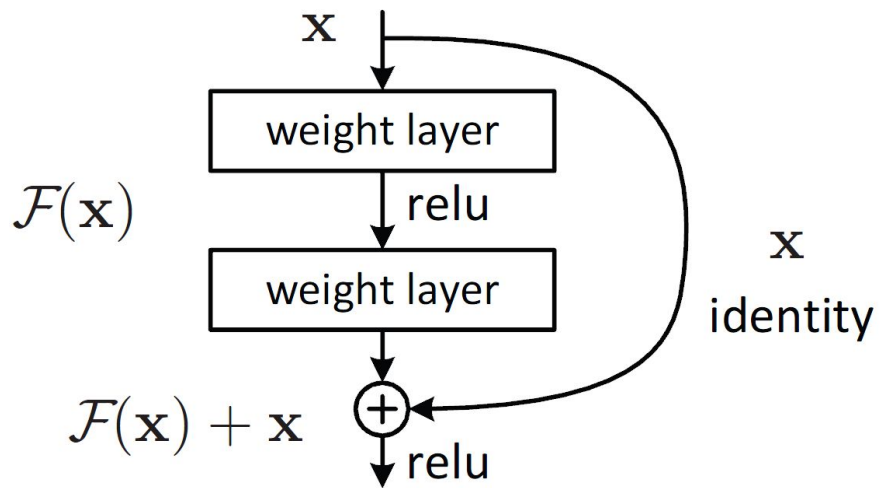
문맥적으로 중요한 단어를 골라내는 것 이외에 새로운 특징을 비선형적으로 조합하고 변환하는 역할 필요

Residual Connection

Attention이나 FFNN의
출력에 입력 x 를 더해줌

정보 손실 방지

Gradient Vanishing 방지





Layer Normalization

Residual Connection은 입력 값을 더하기 때문에 출력 스케일이 폭주할 수 있음

각 토큰 벡터의 값을 일정하게 유지, 학습 속도와 안정성을 높임

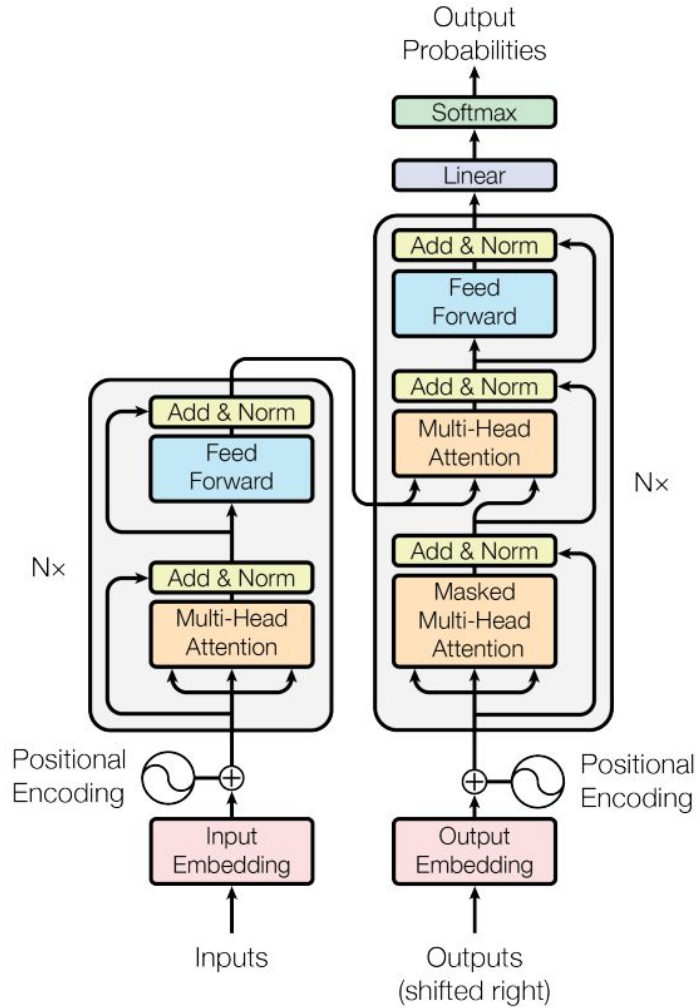


Figure 1: The Transformer - model architecture.